

Guia de testes entre máquinas: macOS + Ubuntu

T1 Laboratório de Redes · Servidor HTTP/1.1 sobre sockets TCP · Grupo 5 · Wi-Fi de casa, IPv4

Papéis: o **Mac** é o **servidor** e faz as capturas (`tshark` + `metricas.py`). O **Ubuntu** é o **cliente** (navegador, `curl` e scripts). Na Fase 6, o **celular** na mesma Wi-Fi é a segunda máquina cliente. Troque **<IP_MAC>** pelo IPv4 do Mac (ex.: `192.168.15.9`), sem colchetes.

MAC Terminal do Mac **UBUNTU** Terminal do Ubuntu **CELULAR** navegador do celular

A rede de casa também tem IPv6. **Use sempre o IPv4** do Mac nos comandos, para que todas as capturas e o relatório fiquem em IPv4.

Fase 0: Preparação (uma vez)

UBUNTU Instalar as ferramentas (o Python 3 já vem no Ubuntu):

```
sudo apt update && sudo apt install -y git curl
```

UBUNTU Baixar o projeto (o repositório é público, não precisa de login):

```
git clone https://github.com/enzotonatto/T1-lab.git && cd T1-lab
```

Se já tiver clonado antes, basta `cd T1-lab && git pull`.

AMBOS Conectar os dois notebooks (e o celular) na **mesma Wi-Fi** de casa, a rede principal (não a de visitantes). VPN desligada.

Fase 1: Verificação inicial (antes de tudo)

MAC IPv4 do Mac:

```
ipconfig getifaddr en0
```

UBUNTU IPv4 do Ubuntu (o primeiro endereço da lista):

```
hostname -I
```

UBUNTU Testar o alcance até o Mac:

```
ping -c 4 <IP_MAC>
```

MAC Testar o alcance até o Ubuntu:

```
ping -c 4 <IP_UBUNTU>
```

Sem resposta? Confira se os dois estão na mesma rede (os três primeiros números do IP iguais, ex.: `192.168.15.x`). Rede “de visitantes” do roteador costuma isolar os aparelhos: use a rede principal.

Fase 2: Subir o servidor

MAC Iniciar o servidor salvando o log (evidência do item 5):

```
cd ~/Developer/T1-lab && python3 server.py --port 8080 --root ./www | tee capturas/servidor.log
```

Se o macOS perguntar se o python3 pode *aceitar conexões de entrada*, clique em **Permitir**. Deixe este terminal aberto e use **outro terminal do Mac** para as capturas.

UBUNTU Teste rápido (deve vir `HTTP/1.1 200 OK`, e o log do Mac mostra o IP do Ubuntu):

```
curl -i http://<IP_MAC>:8080/
```

Fase 3: Navegador (ensaio da interoperabilidade)

UBUNTU No Firefox, abra `http://<IP_MAC>:8080/`. As duas imagens devem aparecer e as linhas “JavaScript carregado” e “JSON carregado” devem ficar **verdes**. Tire um print. No log do Mac aparecem as conexões paralelas do navegador e, 5 s depois, o fechamento por timeout. **Feche a aba** antes de seguir.

Fase 4: Conformidade e travessia (itens 2 e 3)

UBUNTU

```
scripts/conformidade.sh <IP_MAC> > conformidade.txt
```

```
scripts/travessia.sh <IP_MAC> > travessia.txt
```

Conferir: `conformidade.txt` com 200, 200, 400, 400, 403, 404, 405, 405; `travessia.txt` com cinco 403:

```
grep "^HTTP" conformidade.txt travessia.txt
```

Fase 5: Captura de uma transação completa (item 4)

MAC Iniciar a captura:

```
tshark -i en0 -f "tcp port 8080" -w capturas/transacao.pcapng
```

UBUNTU

```
curl -s -o /dev/null http://<IP_MAC>:8080/index.html
```

MAC Esperar 2 s e parar com **Ctrl+C**. Abrir no Wireshark e marcar no print: handshake (SYN, SYN-ACK, ACK), o pacote com o `GET`, os pacotes da resposta e o encerramento (FINs).

Fase 6: Atendimento simultâneo (item 5)

O enunciado pede **duas máquinas clientes ao mesmo tempo**: o Ubuntu e o celular.

MAC Iniciar a captura:

```
tshark -i en0 -f "tcp port 8080" -w capturas/simultaneo.pcapng
```

UBUNTU Abre uma requisição **lenta** (uma linha de cabeçalho por segundo, durante 15 s) e faz uma requisição rápida por segundo:

```
scripts/simultaneo.sh <IP_MAC>
```

CELULAR Durante esses 15 s, abrir `http://<IP_MAC>:8080/` e recarregar a página 2 ou 3 vezes.

MAC Quando o script terminar ("requisição lenta concluída"), **Ctrl+C** no `tshark`.

Evidência: (1) trecho do `capturas/servidor.log` em que a conexão lenta do Ubuntu está aberta e, no meio dela, aparecem conexões do IP do celular, em threads diferentes; (2) print do Wireshark em **Statistics** →

Conversations → **TCP**, com conexões dos dois IPs sobrepostas no tempo (*Rel Start* e *Duration*). A saída do script também mostra que as requisições rápidas foram atendidas em poucos ms enquanto a lenta continuava pendente.

Fase 7 e 8: RTT + medições C1 e C2 (itens 6 a 9), 3 rodadas

Para cada rodada `N` = 1, 2, 3, na ordem abaixo. Nada mais acessando o servidor (navegador e celular fechados).

① **UBUNTU** **RTT** (50 pings a cada 0,2 s; no fim mostra mínimo, **mediana**, média e máximo):

```
scripts/rtt.sh <IP_MAC> | tee ping_N.txt
```

② **MAC** Captura do C1:

```
tshark -i en0 -f "tcp port 8080" -w capturas/c1_N.pcapng
```

③ **UBUNTU** C1: um único `curl`, uma conexão nova por requisição (todas as linhas com `conexões novas` 1):

```
scripts/c1.sh <IP_MAC>
```

④ **MAC** Esperar 2 s e **Ctrl+C**.

⑤ **MAC** Captura do C2:

```
tshark -i en0 -f "tcp port 8080" -w capturas/c2_N.pcapng
```

⑥ **UBUNTU** C2: uma única conexão persistente (`conexões novas` 1 só na primeira linha, depois 0):

```
scripts/c2.sh <IP_MAC>
```

⑦ **MAC** Esperar 2 s e **Ctrl+C**. Repetir ① a ⑦ com `N` = 2 e `N` = 3.

Troque o `N` pelo número da rodada nos nomes: `ping_1.txt`, `c1_1.pcapng`, `c2_1.pcapng` ... Se o Wi-Fi cair no meio de uma rodada, refaça a rodada inteira.

Fase 9: Juntar tudo no Mac e extrair as métricas

UBUNTU Copiar os arquivos de texto para o Mac. Opção sem configurar nada: servir a pasta pela rede por um minuto (é só para transferir; depois feche com Ctrl+C):

```
python3 -m http.server 9000 --bind 0.0.0.0
```

MAC Baixar os arquivos do Ubuntu:

```
for f in conformidade.txt travessia.txt ping_1.txt ping_2.txt ping_3.txt; do curl -s0 "http://<IP_UBUNTU>:9000/$f"; done
```

MAC Parar o servidor (Ctrl+C no terminal dele) e extrair as métricas das 6 capturas:

```
for f in capturas/c1_*.pcapng capturas/c2_*.pcapng; do python3 scripts/metricas.py "$f"; done
```

O `http.server` do Python aqui é só uma ferramenta de cópia entre os notebooks. Ele **não** faz parte do trabalho; o servidor do trabalho continua sendo o `server.py`, sem nenhuma biblioteca HTTP.

Fase 10 (opcional): Inverter os papéis

Ensaia o servidor rodando no Ubuntu e acessado pelo navegador do Mac.

UBUNTU Ver se o firewall está ativo (normalmente “inativo”):

```
sudo ufw status
```

Se estiver ativo, liberar a porta:

```
sudo ufw allow 8080/tcp
```

UBUNTU Subir o servidor:

```
python3 server.py --port 8080 --root ./www
```

MAC No navegador, abrir `http://<IP_UBUNTU>:8080/`.

Se algo falhar

Sintoma	Causa provável / solução
<code>ping</code> funciona, mas o <code>curl</code> dá <i>Connection refused</i>	Servidor parado ou porta errada.
<code>curl</code> fica parado até dar <i>timeout</i>	Firewall do Mac bloqueando o Python: Ajustes do Sistema → Rede → Firewall → Opções → permitir python3.
<code>ping</code> não responde em nenhum sentido	Redes diferentes, ou rede de visitantes com isolamento de aparelhos.
O <code>tshark</code> do Mac não captura nada	Interface errada: <code>tshark -D</code> e usar a da Wi-Fi. Ou o cliente usou o IPv6 em vez do IPv4.
<code>scripts/...: Permission denied</code> no Ubuntu	Rodar <code>chmod +x scripts/*.sh scripts/*.py</code> .
O IP do Mac mudou	O roteador renovou o DHCP. Rode <code>ipconfig getifaddr en0</code> de novo e refaça a rodada em andamento.

Checklist de evidências para o relatório

Item	Evidência	Fase
2 – Conformidade	<code>conformidade.txt</code>	4
3 – Travessia	<code>travessia.txt</code>	4
4 – Transação completa	<code>capturas/transacao.pcapng</code> + print do Wireshark anotado	5
5 – Atendimento simultâneo	trecho de <code>capturas/servidor.log</code> (Ubuntu + celular) + <code>capturas/simultaneo.pcapng</code> (Conversations)	6
6 – RTT	<code>ping_1.txt</code> ... <code>ping_3.txt</code> (mediana)	7
7 a 9 – C1 × C2	<code>capturas/c1_N.pcapng</code> , <code>capturas/c2_N.pcapng</code> + saída do <code>metricas.py</code>	8 e 9
Interoperabilidade	print do navegador com a página completa	3